

PROJECT REPORT

SMART LOST & FOUND MANAGEMENT SYSTEM

Created By:

Akarsh Shukla

Registration Number:

25BAI10715

Course: Programming in Java

Program: B.Tech

College: VIT Bhopal University

Date of Submission: 18 September 2026

1. Introduction

Smart Lost & Found Management System is a college-focused application designed to organize the reporting, searching, matching, and claiming of lost and found items. Instead of relying on notices, informal messages, or separate records, the system provides a centralized platform for students and administrators.

The project combines a modern responsive web interface with a Java backend. The Java backend uses Servlets, JDBC, MySQL, Apache Tomcat, service classes, DAO classes, model classes, and utility classes. The frontend provides pages for registration, login, reporting items, searching, match results, claims, student dashboard, and administration.

A key feature of the project is a Smart Weighted Matching Engine. It compares lost and found records using seven criteria—category, item name, location, color, brand, date proximity, and description similarity—to produce a match score and match category.

The project therefore connects a practical campus problem with concepts learned in Programming in Java, including classes and objects, inheritance, encapsulation, polymorphism, exception handling, packages, collections, JDBC, and web-based request processing.

2. Problem Statement

In a college campus, students may lose items such as identity cards, books, bags, electronic devices, stationery, or personal belongings. Found items may be reported separately, making it difficult to identify whether a found item belongs to a particular student.

Manual methods often make searching slow and depend on incomplete descriptions. A student may know the approximate location, color, category, or date of the lost item but may not know whether someone has reported it as found. The project aims to solve this problem through a centralized lost-and-found system where records can be submitted, searched, automatically compared, and claimed.

The Java implementation further demonstrates how object-oriented programming and database-backed services can be applied to a real-world campus information system.

3. Objectives

- To develop a centralized smart lost-and-found management system for a college campus.
- To allow students to register and log in to the system.
- To provide separate reporting for lost and found items.
- To store item information such as name, category, color, brand, description, location, date, and status.
- To provide item searching and filtering functionality.
- To automatically compare lost and found records using a weighted matching algorithm.
- To display high, medium, low, and unlikely match categories based on the calculated score.
- To allow students to submit claims for relevant items.
- To provide administrators with controls for users, items, matches, and claims.
- To apply Java Servlets, JDBC, MySQL, packages, services, DAO classes, and object-oriented programming.
- To provide validation, password handling, sessions, and controlled error handling.
- To maintain a modular structure that can be extended in the future.

4. Functional Requirements

The system consists of several connected functional modules.

4.1 User Registration and Login

- Student registration using name, email, phone, and student ID.
- Login and session-based access.
- Role distinction between STUDENT and ADMIN.
- Password handling through PasswordUtil.

4.2 Lost Item Management

- Report a lost item.
- Store item name, category, color, brand, description, location, and date.
- Maintain item status such as PENDING, APPROVED, CLAIMED, RETURNED, or REJECTED.
- Associate the report with the user who submitted it.

4.3 Found Item Management

- Report an item that has been found.
- Store descriptive and location information for the found item.
- Allow the record to be processed and compared with lost-item records.
- Maintain the item's current status.

4.4 Search and Match Management

- Search available lost and found records.
- Compare a lost item with a found item.
- Calculate a weighted match score.
- Show matched fields and score breakdown.
- Classify results as High Match, Medium Match, Low Match, or Unlikely Match.

4.5 Claims Management

- Students can submit a claim for an item.
- Claims contain a claimant, item, message, and status.
- Administrators can review claims.
- Claim status can be PENDING, APPROVED, or REJECTED.

4.6 Administration

- Admin dashboard for system-level monitoring.
- Management of users and item records.
- Review of matches and claims.
- Display statistics such as total users, lost items, found items, pending claims, successful returns, and possible matches.

4.7 Smart Matching Engine

The matching engine uses seven weighted criteria: Category 20%, Item Name 20%, Location 20%, Color 10%, Brand 10%, Date Proximity 10%, and Description Similarity 10%. The resulting score is limited to 100 and mapped to a match category.

5. Non-Functional Requirements

5.1 Usability

The interface should allow students and administrators to navigate between reporting, searching, matching, claims, and dashboard functions without unnecessary complexity.

5.2 Performance

The system should provide responsive interaction and efficient database operations for normal campus-scale usage.

5.3 Reliability

Invalid input and database-related operations should be handled without causing the complete application to fail.

5.4 Maintainability

The Java implementation separates models, services, DAO classes, controllers, and utilities. This makes individual components easier to modify and maintain.

5.5 Security and Validation

The project includes password utility and validation classes, JDBC PreparedStatement-based database access, and session-based request handling. These mechanisms support controlled input processing and authenticated access.

5.6 Modularity

Different responsibilities are represented through separate Java classes and packages, while the frontend is divided into reusable React components.

6. System Architecture

The project follows a layered full-stack architecture. The main flow is:

Frontend UI → Java Servlets / Controllers → Service Layer → DAO Layer → MySQL Database

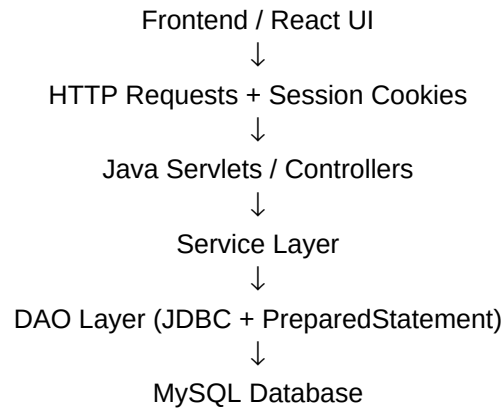
Frontend components include HomePage, RegisterPage, LoginPage, ReportItemPage, SearchPage, MatchResultsPage, ClaimsPage, StudentDashboard, AdminDashboard, and supporting navigation/code-view components.

The Java backend contains the following major layers:

- Model: User, Student, Admin, Item, LostItem, FoundItem, Match, and Claim.
- Utility: DatabaseConnection, PasswordUtil, and ValidationUtil.
- DAO: UserDAO, ItemDAO, MatchDAO, and ClaimDAO.
- Service: MatchingService, LoginService, UserService, ItemService, and ClaimService.
- Controller: LoginServlet, RegisterServlet, LostItemServlet, FoundItemServlet, SearchServlet, MatchServlet, ClaimServlet, and AdminServlet.

The database schema contains tables for users, admin information, items, matches, and claims. JDBC is used to communicate between the Java application and MySQL.

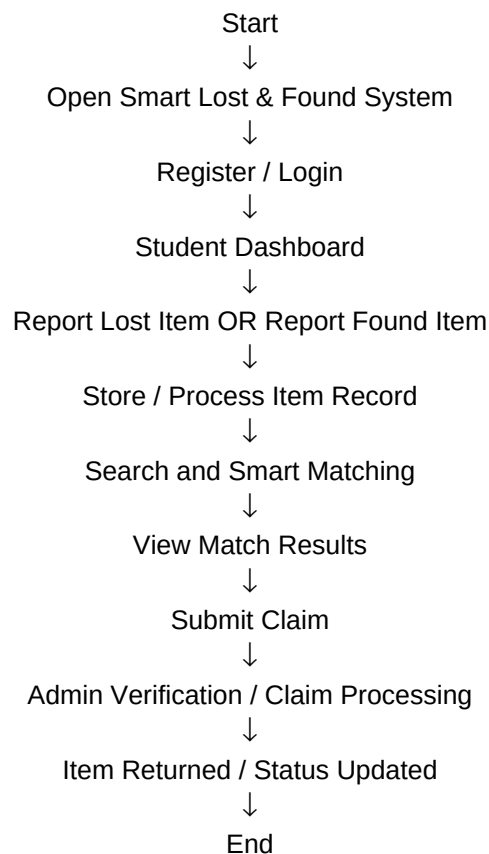
6.1 Architecture Diagram



The architecture separates presentation, request handling, business logic, and database operations. This makes the system easier to understand and maintain.

7. Process Workflow

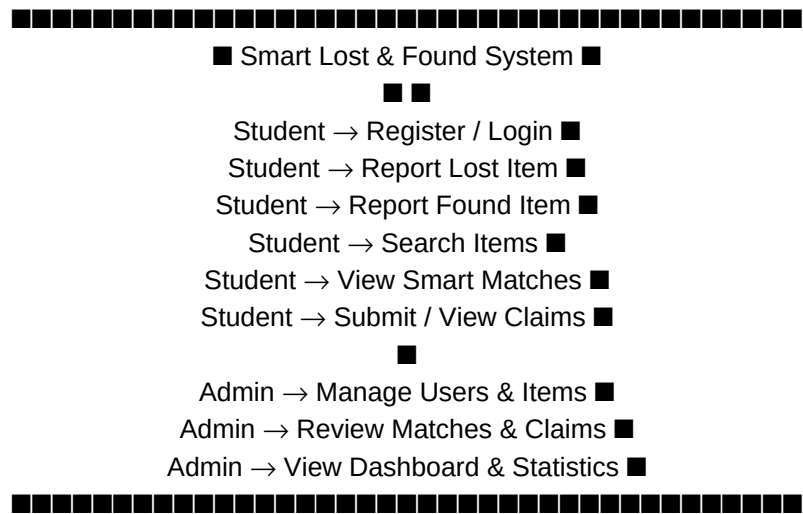
The basic workflow of the system is:



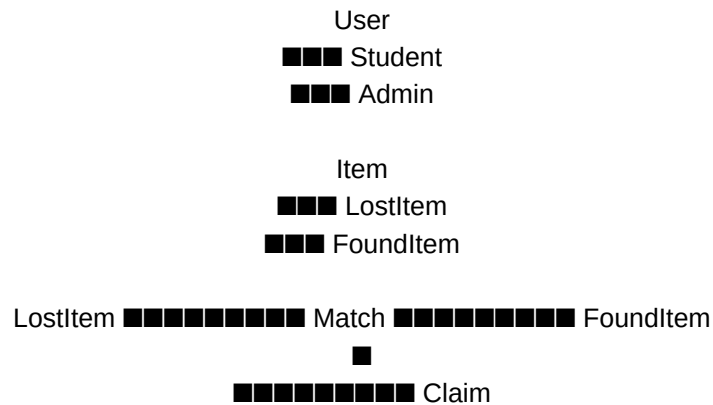
For a match operation, the system retrieves the relevant lost and found records, calculates the seven weighted criteria, produces a score, and classifies the result. The client-side matching engine mirrors the Java matching logic for immediate UI response and verification.

8. Design Diagrams

8.1 Use Case Diagram

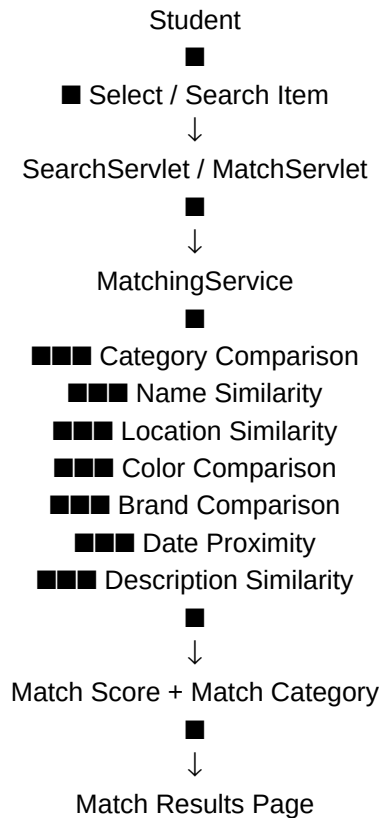


8.2 Class Diagram

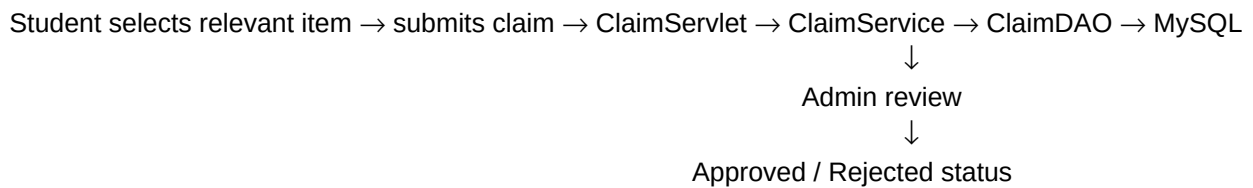


The model layer uses inheritance for user and item specializations. Match represents a relationship between a lost item and a found item, while Claim represents a student's request related to an item.

8.3 Sequence Flow – Smart Matching



8.4 Claim Flow



9. Java Concepts Implemented

Java Concept	Implementation
Classes & Objects	User, Student, Admin, Item, LostItem, FoundItem, Match, Claim and service/DAO/controller classes
Variables & Data Types	Fields using String, int, boolean and numeric types
Methods	Business operations, getters, setters, validation and service methods
Constructors	Used for creating model objects
this Keyword	Used inside class implementations
Encapsulation	Private fields with controlled access through methods
Inheritance	Student/Admin extend User; LostItem/FoundItem extend Item
Polymorphism	Parent-type references can represent specialized objects
Exception Handling	Java exception handling around database and application operations
Packages	com.lostfound.model, util, dao, service and controller
Collections	Used where multiple records/results are processed
JDBC	Database connectivity and SQL operations

Java Concept	Implementation
Servlets	HTTP request handling for application functions

10. Implementation Details

10.1 Model Package

The model package represents the main entities of the application. User stores common account information, while Student and Admin represent roles. Item stores common item attributes, with LostItem and FoundItem representing item types. Match stores the relationship and calculated score between two item records, and Claim stores a user's claim request.

10.2 Service Package

- MatchingService implements the seven-criterion weighted matching engine.
- LoginService handles login-related business operations.
- UserService manages user-related operations.
- ItemService handles item operations.
- ClaimService handles claim-related business logic.

10.3 DAO Package

The DAO layer provides database operations for users, items, matches, and claims. PreparedStatement-based JDBC operations are used to communicate with MySQL.

10.4 Controller Package

- LoginServlet processes login requests.
- RegisterServlet processes registration requests.
- LostItemServlet handles lost-item reporting.
- FoundItemServlet handles found-item reporting.
- SearchServlet handles search operations.
- MatchServlet handles match requests.
- ClaimServlet handles claims.
- AdminServlet provides administrative operations.

10.5 Utility Package

- DatabaseConnection manages database connectivity.
- PasswordUtil provides password-related utility operations.
- ValidationUtil provides validation support.

10.6 Smart Matching Algorithm

The matching engine calculates a score from seven criteria. Category contributes up to 20 points, item name up to 20, location up to 20, color up to 10, brand up to 10, date proximity up to 10, and description similarity up to 10.

Text similarity uses exact/substring checks and token-based Jaccard-style comparison. Date proximity decreases as the difference between incident dates increases. Description comparison removes common stop words before measuring keyword overlap.

10.7 Match Thresholds

Score Range	Match Category
80% – 100%	High Match
60% – 79%	Medium Match
40% – 59%	Low Match
Below 40%	Unlikely Match

The match result also records the fields that contributed to the match and can expose a score breakdown for category, name, location, color, brand, date, and description.

10.8 Frontend Implementation

The frontend is implemented with a React-based component structure and Vite tooling. Important components include the home page, registration, login, reporting, search, match results, claims, student dashboard, and admin dashboard. The `JavaCodeViewerModal` component provides a way to display Java implementation details in the interface.

The frontend data types define `User`, `Item`, `MatchResult`, `Claim`, and `AdminStats` structures. A client-side matching engine mirrors the Java `MatchingService` logic to provide immediate match calculations and verification.

11. Design Decisions and Rationale

- A layered architecture was selected so that presentation, business logic, database operations, and request handling remain separated.
- Separate model classes were used because users, items, matches, and claims have different responsibilities.
- `LostItem` and `FoundItem` inherit common properties from `Item` to avoid unnecessary duplication.
- DAO classes isolate database code from business logic.
- Service classes centralize application rules such as matching and claim processing.
- A seven-criterion weighted algorithm was selected to make matching more informative than relying on only item names.
- Status fields were used to represent the lifecycle of item and claim records.
- MySQL was selected as the relational data store for persistent records.

12. Results / Output

The completed project provides a centralized interface for managing campus lost-and-found activities. Students can register, log in, report lost or found items, search records, view smart match results, and submit claims.

The Smart Matching Engine produces a numerical score and a category for each comparison. It also identifies matched fields, helping users understand why two records were considered similar.

The administrative side provides monitoring and management functionality for users, items, matches, and claims. Statistics include total users, lost items, found items, pending claims, successful returns, and possible matches.

The Java backend demonstrates practical use of Servlets, JDBC, model classes, services, DAO classes, utility classes, inheritance, encapsulation, polymorphism, collections, validation, and exception handling.

The project can be built as a Maven-based Java web application and deployed using Apache Tomcat. The frontend is designed to run as a Vite/React application.

13. Testing Approach

Testing was considered at both the frontend and Java backend levels.

13.1 Frontend Testing

- Landing page and navigation.
- Registration and login page behavior.
- Lost-item and found-item reporting forms.
- Search and filter interactions.
- Match result display and score breakdown.
- Claims page and dashboard navigation.
- Student and admin dashboard components.
- Responsive and interactive UI elements.

13.2 Java Backend Testing

- Creating and using User, Student, Admin, Item, LostItem, FoundItem, Match, and Claim objects.
- Testing inheritance between common and specialized model classes.
- Testing service-layer operations.
- Testing DAO/database operations.
- Testing validation and password utilities.
- Testing matching criteria and threshold categories.
- Testing claim status processing.
- Testing servlet request handling and session-based access.

13.3 Example Test Cases

Test Case	Expected Result
Student registration with valid data	Account is created successfully
Login with valid credentials	User is authenticated and dashboard is accessible
Report a lost item	Lost-item record is stored
Report a found item	Found-item record is stored
Search by item information	Relevant records are displayed
Exact category match	Category contributes full weighted score
Similar item names	Name similarity contributes to score
Same location	Location similarity contributes to score
Same color and brand	Corresponding weighted points are added
Close incident dates	Date proximity score is higher
Similar descriptions	Description similarity contributes to score
Submit a claim	Claim is stored with PENDING status
Admin approves claim	Claim status changes to APPROVED
Invalid input	Validation prevents inappropriate processing

14. Challenges Faced

- Designing a single workflow that connects lost-item reporting, found-item reporting, search, matching, and claims.
- Maintaining consistent item and claim status transitions.
- Connecting the React frontend with a Java Servlet/JDBC backend.
- Organizing the Java project into model, utility, DAO, service, and controller packages.
- Implementing a meaningful matching algorithm instead of relying on one attribute.
- Handling differences in text such as item names, descriptions, and locations.
- Making the match score understandable through matched fields and score breakdown.
- Maintaining validation and session-based access while keeping the application usable.
- Connecting a practical campus problem with core Programming in Java concepts.

15. Learnings and Key Takeaways

- Understanding how Java classes and objects can represent real-world entities.
- Applying encapsulation to protect model data.
- Using inheritance for related user and item types.
- Understanding polymorphism through parent and specialized classes.
- Using packages to organize a larger Java project.
- Using Servlets for web request handling.
- Using JDBC and DAO classes for database interaction.
- Applying service-layer business logic.
- Using collections and data structures for records and results.
- Implementing validation and controlled error handling.
- Designing a weighted algorithm for practical record matching.
- Understanding how frontend components and backend services can work together.
- Improving modularity, maintainability, and organization of source code.

The project also provided practical experience in designing a user interface for a campus-oriented software application.

16. Future Enhancements

Smart Lost & Found Management System can be expanded further through the following enhancements:

Database and Cloud Integration

The system can be deployed on a cloud-hosted MySQL-compatible database and application server for wider campus access.

Image-Based Matching

Computer-vision techniques could be added to compare uploaded item images alongside the current text-based criteria.

Notifications

Email, SMS, or in-application notifications could inform users when a possible match or claim-status update is available.

Advanced Search

Search can be extended with richer filters for category, location, date range, color, brand, status, and similarity score.

Role-Based Access

More roles such as campus security, department coordinators, or help-desk staff could be introduced with appropriate permissions.

Analytics

Additional analytics could show frequently lost categories, locations with high reporting activity, return rates, and processing times.

Mobile Application

A dedicated Android or iOS application could make reporting and claim management more accessible from mobile devices.

QR / Campus ID Integration

Campus identity or QR-based verification could be integrated to strengthen item handover and claim verification.

Audit Trail

A detailed activity history could record important actions such as item approval, claim decisions, and return confirmation.

17. Conclusion

Smart Lost & Found Management System demonstrates the application of Programming in Java concepts to a practical campus-management problem. The system brings registration, authentication, lost-item reporting, found-item reporting, search, smart matching, claims, and administration into one organized platform.

The Java backend demonstrates object-oriented programming through model classes, inheritance, encapsulation, and polymorphism. It also uses Servlets, JDBC, DAO classes, service classes, utility classes, validation, sessions, and database persistence to create a structured application.

The Smart Weighted Matching Engine is a central feature of the project. By combining category, item name, location, color, brand, date proximity, and description similarity, the system produces a score that can help users identify possible relationships between lost and found records.

The current system provides a foundation that can be extended with image recognition, notifications, cloud deployment, advanced analytics, mobile access, stronger verification, and additional campus roles.

Overall, the project shows how Java programming concepts can be combined with a web interface and relational database to create a complete, modular, and practical software solution.

18. References

- [Oracle Java Documentation](#)
- [Java Servlet / Jakarta Servlet Documentation](#)
- [MySQL Documentation](#)
- [Apache Tomcat Documentation](#)
- [React Documentation](#)
- [Vite Documentation](#)
- [JDBC Documentation](#)
- [MDN Web Docs](#)
- [GitHub Documentation](#)

19. Project Screens / Output

The project source contains frontend components corresponding to the main application screens, including the home page, registration, login, report-item page, search page, match results, claims page, student dashboard, and admin dashboard.

These screens represent the user-facing implementation of the modules described throughout this report. Actual screenshots can be inserted in this section when the application is run locally.

19.1 Suggested Screenshot Placement

Figure	Screen / Output
Figure 1	Home / Landing Page
Figure 2	Student Registration
Figure 3	Login Page
Figure 4	Report Lost / Found Item
Figure 5	Search Items
Figure 6	Smart Match Results
Figure 7	Claims Page
Figure 8	Student Dashboard
Figure 9	Admin Dashboard
Figure 10	Java Code Viewer / Backend Structure

Note: The uploaded project source contains the corresponding React components and Java backend classes. This report intentionally describes the implemented structure and does not invent screenshots that were not provided as image files.

Appendix – Project Technology Stack

Layer	Technology / Implementation
Frontend	React, TypeScript, Vite, HTML/CSS
Backend	Java Servlets
Database	MySQL
Database Access	JDBC, PreparedStatement
Server	Apache Tomcat
Build	Maven
Architecture	Controller → Service → DAO → Database
Matching	7-Criterion Smart Weighted Matching Engine
Main Roles	STUDENT and ADMIN

Prepared for:

Programming in Java

VIT Bhopal University

Akarsh Shukla

Registration Number: 25BAI10715